JC1503: Object-Oriented Programming
Group Project: Personal Finance Tracker

Group [AI Group 20]

May 2, 2026

Individual Contributions

Member	Responsibilities	Technical Implementation
Xinyuan Lin	System architecture, main program logic, data persistence, undo system, report writing, participated in the development of account.py	`main.py`, `linked_list.py`, `transaction.py`, `account.py`, `exceptions.py`; HashMap/BST index integration, bug fixing
Zidan Ye	Account hierarchy design, OOP modelling (inheritance and polymorphism)	`account.py` (Account ABC, SavingsAccount, CreditAccount); code debugging
Yu Chen	Core data structure implementations	`bst.py`, `hashmap.py`, `tree.py`, `set.py`, `stack.py`, `queue.py`
Erhao Yao	Category management, notification system, testing , report writing	`category.py`, `notification.py`; data generation, manual testing

Table 1: Summary of individual member responsibilities.

1 Problem and System Design

1.1 Application Domain

Our system is a **Personal Finance Tracker** that helps individuals manage their daily income and expenditure records. In everyday life, people need to track where their money goes, categorise spending habits, monitor large or unusual transactions, and manage bank account balances—tasks that are tedious and error-prone when done manually. Our application addresses this by providing a structured, persistent tool for recording, searching, and analysing personal financial data.

The system models three core real-world entities and their relationships:

- **Transaction** — an individual income or expense record, linked to a category and optionally to an account.
- **Category** — a hierarchical classification (e.g. *Expense* > *Food* > *Takeaway*), organised as a tree.
- **Account** — a bank account (savings or credit) whose balance is automatically updated when transactions are linked.

These entities are not independent: a transaction belongs to a category and may belong to an account; an account's balance evolves as transactions are added or removed. This interdependency makes the system a coherent whole rather than a loose collection of features.

1.2 System Architecture

The system is organised into three layers, as shown in Figure 1.

User Interface Layer (`main.py` — menu-driven CLI)
Business Logic Layer
Transaction CRUD | Category Management | Account Management
Undo (Stack) | Notification (Queue)
Data and Index Layer
DoublyLinkedList | HashMap (tid index) | BST (date/amount index)
Tree + Set (categories) | JSON Persistence

Figure 1: System architecture overview.

Why a system, not a script?

The application maintains persistent state across sessions (JSON files), evolves its data as the user interacts, and its modules interact through shared indices and data structures. A script would simply re-compute results from static inputs; our system loads previous state, allows incremental modification, and saves the updated state—behaving like a real application.

2 Technical Design

2.1 Object-Oriented Modelling

Our design applies all core OOP principles:

Abstraction. `Account` is an abstract base class (inheriting from `ABC`) with two abstract methods—`get_account_type()` and `calculate_interest()`—that every concrete account type must implement. This enforces a consistent interface while allowing type-specific behaviour.

```python
class Account(ABC):
    @abstractmethod
    def get_account_type(self): pass
    @abstractmethod
```

```
    def calculate_interest(self): pass
```

Inheritance. `SavingsAccount` and `CreditAccount` both extend `Account`, inheriting common fields (`account_id`, `owner`, `_balance`) and methods (`deposit`, `get_balance`, `to_dict`). Each subclass adds its own attributes: `interest_rate` for savings and `credit_limit` for credit.

Polymorphism. `calculate_interest()` behaves differently in each subclass: savings accounts return `balance` $\times$ `rate`, while credit accounts return `|overdraft|` $\times$ `0.20` (or 0 if no overdraft). Similarly, `CreditAccount.withdraw()` overrides the base method to allow negative balances up to the credit limit. The call site in `main.py` uses the base-class interface without knowing the concrete type.

Encapsulation. Internal state is hidden using Python's convention: `_balance` is a private attribute accessed only through `get_balance()` and modified only through `deposit()`/`withdraw()`, which validate inputs and raise custom exceptions on errors. Similarly, `Stack._items` and `Queue._items` are not exposed externally.

Composition. `CategoryManager` composes a `Tree` (for hierarchical structure) and a `Set` (for name uniqueness). `NotificationManager` composes a `Queue` (for FIFO message consumption). These has-a relationships are preferred over inheritance because manager classes delegate to their component structures rather than specialising them.

2.2 Data Structures and Algorithms

All six required data structures are **implemented from scratch** (no Python built-in `dict` or `list` used for core logic) and are integrated into the system's behaviour:

Structure	File	Role in System	Complexity
Doubly Linked List	`linked_list.py`	Primary storage for transactions; O(1) head insert/delete	O(1)
HashMap	`hashmap.py`	tid $\rightarrow$ Node index for O(1) lookup	O(1) avg
BST	`bst.py`	Date and amount indices for range queries	O($\log n$) avg
Stack	`stack.py`	Undo operation history (LIFO)	O(1) push/pop
Queue	`queue.py`	Notification FIFO consumption	O(1) enq/deq
Tree (multi-way)	`tree.py`	Hierarchical category structure	—
Set (on HashMap)	`set.py`	Category name uniqueness check	O(1) avg

Table 2: Data structure usage summary.

Recursive algorithms. The BST implements recursive insert, search, delete, in-order traversal, and range query. The Tree implements recursive DFS traversal, height computation, and the CategoryManager uses recursive serialisation. A representative example is the BST range query:

```
def _range_query_recursive(self, node, low, high, result):
    if node is None: return
    if low < node.key:
        self._range_query_recursive(node.left, low, high, result)
    if low <= node.key <= high:
        result.append(node.value)
    if node.key < high:
        self._range_query_recursive(node.right, low, high, result)
```

2.3 Efficiency and Complexity

A key design decision was building auxiliary indices on top of the linked list to avoid $O(n)$ linear scans:

- **HashMap index** (`tid_index`): Finding a transaction by ID is $O(1)$ on average instead of $O(n)$ list traversal. This is critical for update and delete operations.
- **BST indices** (`date_index`, `amount_index`): Range queries run in $O(\log n + k)$ where k is the number of matching records, compared to $O(n)$ for a full scan.
- **Trade-off**: Building indices takes $O(n \log n)$ at startup, but this is a one-time cost that pays off across all subsequent queries.
- **Limitation**: The BST is not self-balancing; in the worst case it degrades to a linked list with $O(n)$ search time. An AVL or red-black tree would guarantee $O(\log n)$ worst-case performance.

3 Development and Testing

3.1 Iterative Development

The system was built in four stages:

1. **Foundation**: Doubly linked list plus Transaction model plus basic CRUD (add/delete/view). Data persisted via JSON with `to_dict()`/`from_dict()` class methods.
2. **Indexing**: Added HashMap (tid index) and BST (date/amount indices) to optimise lookup and range queries from $O(n)$ to $O(1)$ and $O(\log n)$.
3. **Support features**: Added Stack-based undo system, Queue-based notification system, Tree plus Set category management with recursive serialisation.
4. **Account system**: Introduced `Account` abstract class with `SavingsAccount` and `CreditAccount` subclasses, demonstrating inheritance, polymorphism, and custom exceptions.

Each stage added functionality that was tested manually and then integrated into the growing system.

3.2 Problem Solving

Problem 1: JSON-to-Object Conversion. When loading data from JSON, we needed to convert plain dictionaries back into `Transaction` and `Account` objects. Initially we used standalone functions, but this scattered creation logic away from the class definition. The solution was to use `@classmethod` factory methods:

```
@classmethod
def from_dict(cls, data: dict) -> 'Transaction':
    return cls(tid=data["tid"], type=data["type"],
               amount=data["amount"], ...)
```

This keeps serialisation and deserialisation logic paired with the class, improving cohesion and maintainability.

Problem 2: Transaction Lookup in Linked List Another problem we ran into was with the doubly linked list. Since all transactions were stored in the list, any operation that needed to find a specific transaction by its ID required traversing the list from the beginning. In the worst case, this took O(n) time, which felt inefficient. Yu Chen had already implemented a HashMap for another part of the project, so we decided to use it here as well. We used the transaction ID as the key and mapped it directly to the node reference in the linked list. After adding this index, finding a transaction by ID became O(1) on average, which made operations like update and delete much faster.

Problem 3: Transaction ID Uniqueness. Allowing duplicate tids would corrupt the HashMap index. We solved this with a validation loop in `main.py` that uses `check_tid_exists()` (O(1) via HashMap) before accepting a new transaction, prompting the user to re-enter if a duplicate is detected.

Problem 4: Undo State Management.(Command Pattern) A serious problem we encountered was figuring out where the stack data structure could actually be applied in our system. After searching for some material online, we decided to use it for an undo feature, following the Command pattern. Recording enough information to reverse any operation was not trivial. The Stack stores operation descriptors—action type, transaction ID, and the old data—so that the reverse operation can be computed: undoing an add means deleting that transaction; undoing a delete means re-adding it with the saved data; undoing an update means restoring the original field values. This design basically encapsulates each user action as a command object that knows how to reverse itself. By pushing these command objects onto a LIFO Stack, we keep the CLI separate from the data layer, which also makes it easier to add things like redo or macro commands later.

Problem 5: Transaction Rollback on Insufficient Balance. During testing, we discovered a subtle bug: when a user attempted to withdraw more than the available balance from a savings account, the system correctly raised an `InsufficientBalanceError`. However, the transaction record had already been added to the linked list and indices before the balance check occurred. This meant that failed withdrawals were still being saved and displayed, corrupting the user's financial history.

To fix this, we reordered the operation sequence. The system now performs the balance validation *before* committing the transaction to storage. If the account has insufficient funds, the operation aborts immediately without modifying any data structures. This ensures that the transaction list and account balances remain consistent—a failed operation leaves no trace, which is the expected behaviour for our program.

3.3 Testing Strategy

Unit Testing. Each module file includes an `if __name__ == "__main__"` block at the bottom that exercises the module's key operations. For example, `account.py` tests deposit, withdraw (including overdraft), interest calculation, and custom exception handling; `bst.py` tests insert, search, range query, and delete; `hashmap.py` tests put, get, update, and remove.

System Testing. We performed end-to-end manual testing by running the full application and verifying:

- Transaction CRUD: Creating, updating (partial field update via HashMap O(1) index), and deleting transactions with duplicate-tid rejection and transaction-account linkage; sorting by tid ascending or date descending.
- BST range queries:Range queries by date and by amount using two BST indexes function(build date index/build amount index), returning correct results with O(log n) efficiency.
- Undo system (Stack/LIFO) undo operations covering transactions , category (delete with subtree snapshot), and account , with peek-and-confirm before execution.
- Category management (Tree + Set + JSON):Multi-level category hierarchy (Tree), unique name enforcement (Set based on HashMap), category CRUD with associated-transaction preview on delete, and per-category statistics.
- Account balance updating correctly when transactions are linked.

- Notifications system using queue, welcome info and large expense notifications,etc. FIFO read with unread count and read history tracking.
- Display transactions connected to any account ID correctly.
- Data Persistence (JSON):Three file persistence for transactions (DoublyLinkedList), categories (Tree), and accounts(HashMap) with to-dict() / from-dict() conversion.
- The program not crashing on malformed input (negative amounts, non-numeric IDs).

4 Reflection

4.1 Critical Assessment

What works well. The choice of data structures maps naturally to the domain: a linked list for sequential transaction storage, a HashMap for instant ID lookup, BSTs for range queries, a Tree for hierarchical categories, a Stack for undo, and a Queue for notifications. The OOP design cleanly separates concerns. Moreover, the abstract `Account` class with polymorphic `calculate_interest()` makes it straightforward to add new account types in the future.

Current limitations.

- The BST is not self-balancing; sorted insertions degrade it to $O(n)$ search time.
- Interest calculation does not consider the time period (days/months); it always computes annual interest.
- Data is stored in plain JSON with no encryption or access control.
- The CLI interface, while functional, is not intuitive for non-technical users.

4.2 Future Improvements

If development continued, we would:

- Replace the plain BST with an AVL tree or red-black tree for guaranteed $O(\log n)$ worst-case performance.(P2)
- Add a `days` parameter to `calculate_interest()` so that users can compute interest for arbitrary periods.(P3)
- Implement a GUI (e.g. using Tkinter or a web framework) for better usability.(P1)
- Add password protection and data encryption for stored files.(P1)
- Implement budget tracking and spending trend visualisation.(P2)

4.3 Learning Outcomes

Through this project, we gained practical understanding of several key principles:

- **Composition over inheritance**: `CategoryManager` composes a `Tree` and a `Set` rather than inheriting from either, because it needs both behaviours simultaneously. This is cleaner than multiple inheritance.
- **Data structure choice matters**: Building HashMap and BST indices transformed $O(n)$ operations into $O(1)$ and $O(\log n)$, making the system responsive even with many records.
- **Recursion is natural for trees**: Traversal, serialisation, and height computation all follow the recursive structure of the tree itself, producing concise and readable code.
- **Iterative refinement**: The system grew through four stages, with each stage adding and testing functionality before proceeding. Problems discovered during testing led to further design improvements.
- **Object-oriented data handling**: When reading transaction records from JSON files, we mapped dictionary data to object instances. We discovered that encapsulating this logic within class methods significantly streamlined the process, reducing boilerplate code and improving maintainability.

- **What we would do differently**: Looking back, we should have written more automated tests earlier. Most of our testing was manual, and catching bugs late in the integration phase cost us extra time.